```
require("dotenv").config();
const express = require("express");
const path = require("path");
const fs = require("fs");
const { Pool } = require("pg");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const QRCode = require("qrcode");
const { AsyncLocalStorage } = require("async_hooks");
const tenantContext = new AsyncLocalStorage();

const app = express();
app.set("trust proxy", 1);
app.use(express.json({limit:"2mb"}));
app.use(express.urlencoded({extended:true}));
// Frontend is served explicitly from the absolute filesystem path.
// This avoids any dependency on Express static-directory index resolution.
const PUBLIC_DIR = path.resolve(__dirname, "public");
const INDEX_HTML = path.join(PUBLIC_DIR, "index.html");
const INDEX_CONTENT = fs.readFileSync(INDEX_HTML, "utf8");

app.get(/^\/$/, (req, res) => {
  res.status(200).type("html").send(INDEX_CONTENT);
});
app.get(/^\/index\.html\/?$/, (req, res) => {
  res.status(200).type("html").send(INDEX_CONTENT);
});

// Static assets (logo, icons, manifest, service worker, etc.).
app.use(express.static(PUBLIC_DIR, { index: false }));
app.use("/api", (req,res,next)=>{ res.set("Cache-Control","no-store"); next(); })
app.use((req,res,next)=>{ res.set("X-Content-Type-Options","nosniff"); res.set("R

const pool = new Pool({connectionString: process.env.DATABASE_URL});

// Every authenticated request carries its tenant in AsyncLocalStorage.
// The query wrapper sets PostgreSQL session context on the same connection,
// allowing Row-Level Security to enforce tenant isolation server-side.
const rawPoolQuery = pool.query.bind(pool);
pool.query = async function tenantAwareQuery(text, params) {
  const ctx = tenantContext.getStore();
  if (!ctx?.shopId) return rawPoolQuery(text, params);
  const client = await pool.connect();
  try {
```

```
    await client.query("BEGIN");
    await client.query("SELECT set_config('app.shop_id',$1,true)",[String(ctx.sho
    const result = await client.query(text, params);
    await client.query("COMMIT");
    return result;
  } catch (e) {
    try { await client.query("ROLLBACK"); } catch {}
    throw e;
  } finally { client.release(); }
};

async function setTenant(client, shopId){
  await client.query("SELECT set_config('app.shop_id',$1,true)",[String(shopId)])
}
const JWT_SECRET = process.env.JWT_SECRET || "dev-only-change-me";
const PORT = Number(process.env.PORT || 3000);
const DEFAULT_ADMIN_PASSWORD = process.env.DEFAULT_ADMIN_PASSWORD || "Admin@12345

const DEFAULT_PERMS = {
  add_repair:true, view_repairs:true, edit_repair:true, change_status:true,
  customers:true, edit_customers:true, inventory:true, accounts:false,
  profits:false, reports:false, delete_repairs:false, delete_customers:false,
  users:false, settings:false, activity_logs:false, backups:false,
  view_repair_count:true, view_in_repair:true
};
const STATUSES = ["لايصلح","تم اصلاحه","قيد الاصلاح"];

async function q(text, params=[]){ return (await pool.query(text,params)).rows; }
async function one(text, params=[]){ return (await pool.query(text,params)).rows[
async function init(){
  const schemaCandidates = [
    path.join(__dirname, "db", "schema.sql"),
    path.join(__dirname, "BN-SMART-schema.sql"),
    path.join(__dirname, "src", "db", "schema.sql"),
  ];
  const schemaPath = schemaCandidates.find((p) => fs.existsSync(p));
  if (!schemaPath) {
    throw new Error(`Database schema not found. Checked: ${schemaCandidates.join(
  }
  const schema=fs.readFileSync(schemaPath,"utf8");
  await pool.query(schema);

  // SaaS foundation: create the tenant table and migrate legacy single-shop data
  // into one default tenant before enabling RLS.
  await pool.query(`CREATE TABLE IF NOT EXISTS shops (
    id BIGSERIAL PRIMARY KEY, slug VARCHAR(80) UNIQUE NOT NULL, name VARCHAR(160)
    logo TEXT, phone VARCHAR(40), address TEXT, currency VARCHAR(12) NOT NULL DEF
```

```
  receipt_settings JSONB NOT NULL DEFAULT '{}'::jsonb, created_at TIMESTAMPTZ N
  active BOOLEAN NOT NULL DEFAULT TRUE
)`);
const defaultShop=(await pool.query("SELECT * FROM shops WHERE slug='bn-smart'
  (await pool.query("INSERT INTO shops(slug,name,phone,address,currency) VALUES
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS due_at TIM
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS customer_r
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS received_a
await pool.query(`ALTER TABLE repair_parts ADD COLUMN IF NOT EXISTS sale_price

// Compatibility migration: older deployments may have been created from a sche
// that did not contain supplier_purchases. Create the table before any ALTER/I
// statements so startup is self-healing instead of failing with 42P01.
await pool.query(`CREATE TABLE IF NOT EXISTS supplier_purchases (
  id BIGSERIAL PRIMARY KEY,
  shop_id BIGINT,
  supplier VARCHAR(120) NOT NULL,
  items JSONB NOT NULL DEFAULT '[]'::jsonb,
  total_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  paid_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  notes TEXT,
  created_by BIGINT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
)`);

const tenantTables=['users','customers','repair_orders','repair_status_history'
for(const table of tenantTables){
  await pool.query(`ALTER TABLE ${table} ADD COLUMN IF NOT EXISTS shop_id BIGIN
  await pool.query(`UPDATE ${table} SET shop_id=$1 WHERE shop_id IS NULL`,[defa
  await pool.query(`ALTER TABLE ${table} ALTER COLUMN shop_id SET NOT NULL`);
  await pool.query(`CREATE INDEX IF NOT EXISTS idx_${table}_shop_id ON ${table}
}
await pool.query(`ALTER TABLE supplier_purchases ADD COLUMN IF NOT EXISTS shop_
await pool.query(`UPDATE supplier_purchases SET shop_id=$1 WHERE shop_id IS NUL
await pool.query(`ALTER TABLE supplier_purchases ALTER COLUMN shop_id SET NOT N
await pool.query(`CREATE INDEX IF NOT EXISTS idx_supplier_purchases_shop_id ON
await pool.query(`ALTER TABLE users DROP CONSTRAINT IF EXISTS users_username_ke
await pool.query(`CREATE UNIQUE INDEX IF NOT EXISTS uq_users_shop_username ON u
const fkTables=['users',...tenantTables.filter(t=>t!=='users'),'supplier_purcha
for(const table of fkTables){
  const cname=`fk_${table}_shop`;
  await pool.query(`DO $$ BEGIN IF NOT EXISTS (SELECT 1 FROM pg_constraint WHER
}

// Settings used to be global. Migrate it once to tenant-scoped key/value stora
const settingsHasShop=await pool.query(`SELECT 1 FROM information_schema.column
if(!settingsHasShop.rows.length){
```

```
    await pool.query(`CREATE TABLE IF NOT EXISTS settings_tenant (id BIGSERIAL PR
    const oldSettings=await pool.query("SELECT key,value FROM settings");
    for(const row of oldSettings.rows){ await pool.query("INSERT INTO settings_te
    await pool.query("DROP TABLE settings");
    await pool.query("ALTER TABLE settings_tenant RENAME TO settings");
    await pool.query("CREATE INDEX IF NOT EXISTS idx_settings_shop_id ON settings
  }

  const exists=(await pool.query("SELECT id FROM users WHERE shop_id=$1 LIMIT 1",
  if(!exists){
    const hash=await bcrypt.hash(DEFAULT_ADMIN_PASSWORD,12);
    await pool.query("INSERT INTO users(shop_id,username,full_name,password_hash,
      [defaultShop.id,"admin","المدير الرئيسي",hash,JSON.stringify(Object.fromEnt
  }
  const shopRow=await pool.query("SELECT value FROM settings WHERE shop_id=$1 AND
  const shopValue=shopRow.rows[0]?.value && typeof shopRow.rows[0].value==='objec
  const fixedShop={...shopValue,name:"BN SMART"};
  await pool.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',$2)

  // Enforce tenant isolation in PostgreSQL for every tenant-owned table.
  const rlsTables=['users','customers','repair_orders','repair_status_history','p
  for(const table of rlsTables){
    await pool.query(`ALTER TABLE ${table} ENABLE ROW LEVEL SECURITY`);
    await pool.query(`DROP POLICY IF EXISTS ${table}_tenant_isolation ON ${table}
    await pool.query(`CREATE POLICY ${table}_tenant_isolation ON ${table} USING (
  }

  // Public tracking uses a narrowly-scoped SECURITY DEFINER function. It accepts
  // only a tracking code and returns one repair, while authenticated APIs remain
  await pool.query(`CREATE OR REPLACE FUNCTION public_track_repair(p_code TEXT) R
    id BIGINT, receipt_no INTEGER, tracking_code VARCHAR, brand VARCHAR, model VA
    updated_at TIMESTAMPTZ, due_at TIMESTAMPTZ, expected_price NUMERIC, paid_amou
  ) LANGUAGE SQL SECURITY DEFINER SET search_path=public AS $$
    SELECT r.id,r.receipt_no,r.tracking_code,r.brand,r.model,r.color,r.fault,r.st
    FROM repair_orders r JOIN customers c ON c.id=r.customer_id WHERE r.tracking_
  $$`);
  await pool.query(`REVOKE ALL ON FUNCTION public_track_repair(TEXT) FROM PUBLIC`
  await pool.query(`GRANT EXECUTE ON FUNCTION public_track_repair(TEXT) TO PUBLIC
  await pool.query(`CREATE OR REPLACE FUNCTION public_track_history(p_repair_id B
    SELECT COALESCE(json_agg(x ORDER BY x.created_at),'[]'::json) FROM (SELECT st
  $$`);
  await pool.query(`REVOKE ALL ON FUNCTION public_track_history(BIGINT) FROM PUBL
  await pool.query(`GRANT EXECUTE ON FUNCTION public_track_history(BIGINT) TO PUB
}
function auth(req,res,next){
  const h=req.headers.authorization||"";
  const token=h.startsWith("Bearer ")?h.slice(7):null;
```

```
  if(!token)return res.status(401).json({error:"تسجيل الدخول مطلوب"});
  try{
    req.user=jwt.verify(token,JWT_SECRET);
    if(!req.user.shop_id)return res.status(401).json({error:"لا تحتوي على محل صالح
    tenantContext.run({shopId:req.user.shop_id},()=>next());
  }catch(e){res.status(401).json({error:"الجلسة غير صالحة"})}
}
function manager(req,res,next){ if(req.user.role!=="manager")return res.status(40
function can(permission){return (req,res,next)=>{ if(req.user.role==="manager" ||
function effPerms(u){return u.role==="manager"?Object.fromEntries(Object.keys(DEF
function has(req,perm){return !!effPerms(req.user)[perm]}
async function log(userId,action,repairId=null,details={}){const ctx=tenantContex

app.post("/api/auth/register-shop",async(req,res)=>{
  const b=req.body||{};
  const shopName=String(b.shopName||"").trim();
  const slug=String(b.slug||shopName).trim().toLowerCase().replace(/[^a-z0-9-]+/g
  const username=String(b.username||"admin").trim();
  const password=String(b.password||"");
  if(shopName.length<2 || !slug || username.length<2 || password.length<8) return
  const client=await pool.connect();
  try{
    await client.query("BEGIN");
    const shop=(await client.query("INSERT INTO shops(slug,name,phone,address,cur
    await setTenant(client,shop.id);
    const hash=await bcrypt.hash(password,12);
    const permissions=Object.fromEntries(Object.keys(DEFAULT_PERMS).map(k=>[k,tru
    const user=(await client.query("INSERT INTO users(shop_id,username,full_name,
    await client.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',
    await client.query("COMMIT");
    res.status(201).json({shop:{id:shop.id,slug:shop.slug,name:shop.name},user});
  }catch(e){try{await client.query("ROLLBACK")}catch{}; if(e?.code==='23505') ret
});

app.post("/api/auth/login",async(req,res)=>{
  try{
    const {username,password,shopSlug}=req.body;
    const slug=String(shopSlug||"bn-smart").trim().toLowerCase();
    const shop=await one("SELECT id,slug,name,active FROM shops WHERE slug=$1",[s
    if(!shop || !shop.active) return res.status(401).json({error:"جود أو غير مفعل
    const u=(await pool.query("SELECT * FROM users WHERE shop_id=$1 AND username=
    if(!u || !u.active || !(await bcrypt.compare(password||"",u.password_hash)))
    await tenantContext.run({shopId:shop.id},()=>pool.query("UPDATE users SET las
    const permissions=u.role==="manager"?Object.fromEntries(Object.keys(DEFAULT_P
    const token=jwt.sign({id:u.id,username:u.username,full_name:u.full_name,role:
    res.json({token,user:{id:u.id,username:u.username,full_name:u.full_name,role:
  }catch(e){res.status(500).json({error:"خطأ في الخادم"})}
```

```
});
app.get("/api/auth/me",auth,(req,res)=>res.json({user:{...req.user,permissions:ef
app.get("/health",(req,res)=>res.json({ok:true,service:"BN SMART"}));

app.get("/api/dashboard",auth,can("view_repairs"),async(req,res)=>{
  const stats=await one(`SELECT
    COUNT(*) FILTER(WHERE status='قيد الاصلاح') AS in_repair,
    COUNT(*) FILTER(WHERE status='تم اصلاحه') AS repaired,
    COUNT(*) FILTER(WHERE status='لايصلح') AS not_repairable,
    COUNT(*) FILTER(WHERE created_at::date=CURRENT_DATE) AS received_today,
    COUNT(*) FILTER(WHERE customer_received=TRUE) AS customer_received_count,
    COALESCE(SUM(paid_amount),0) AS revenue,
    COALESCE(SUM(expected_price-part_cost) FILTER(WHERE customer_received=TRUE),0
    COALESCE(SUM(expected_price-paid_amount),0) AS remaining
    FROM repair_orders`);
  const recent=await q(`SELECT r.*,c.name customer_name,c.phone FROM repair_order
  const statuses=await q("SELECT status,COUNT(*) count FROM repair_orders GROUP B
  const st={...stats};let sts=statuses;
  if(!has(req,"profits"))delete st.profit;
  if(!has(req,"accounts"))delete st.remaining;
  if(!has(req,"view_in_repair")){delete st.in_repair;sts=sts.filter(x=>x.status!=
  if(!has(req,"view_repair_count")){for(const k of ["repaired","not_repairable","
  res.json({stats:st,recent,statuses:sts});
});

app.get("/api/reports/daily",auth,can("reports"),async(req,res)=>{
  try{
    const period=String(req.query.period||"30");
    let where="";
    const params=[];
    if(period!=="all"){
      const days=Math.max(1,Math.min(3650,Number(period)||30));
      params.push(days);
      where=`WHERE r.created_at >= CURRENT_DATE - ($1::int - 1) * INTERVAL '1 day
    }
    const rows=await q(`SELECT r.created_at::date AS day, COUNT(*)::int AS repair
      COALESCE(SUM(r.paid_amount),0)::numeric AS paid,
      COALESCE(SUM(r.expected_price-r.part_cost) FILTER (WHERE r.customer_receive
      FROM repair_orders r ${where}
      GROUP BY r.created_at::date ORDER BY repairs_count DESC, day DESC`,params);
    const total=rows.reduce((n,r)=>n+Number(r.repairs_count||0),0);
    const totalPaid=rows.reduce((n,r)=>n+Number(r.paid||0),0);
    const totalProfit=rows.reduce((n,r)=>n+Number(r.profit||0),0);
    const daysSpan=period==="all"?Math.max(rows.length,1):Math.max(1,Number(perio
    res.json({
      total_repairs: total,
      total_paid: totalPaid,
```

```
      ...(has(req,"profits")?{total_profit:totalProfit}:{}),
      average_daily_repairs: total/daysSpan,
      days: rows.map(r=>{const o={...r,percent: total?Number(r.repairs_count)*100
    });
  }catch(e){console.error(e);res.status(500).json({error:"تعذر تحميل تقرير الأيام"
});

app.get("/api/repairs",auth,can("view_repairs"),async(req,res)=>{
  const {search="",status=""}=req.query;
  const params=[];let where=[];
  if(status){params.push(status);where.push(`r.status=$${params.length}`)}
  if(search){params.push(`%${search}%`);where.push(`(r.receipt_no::text ILIKE $${
  const sql=`SELECT r.*,c.name customer_name,c.phone customer_phone,u.full_name c
             FROM repair_orders r JOIN customers c ON c.id=r.customer_id LEFT JOI
             ${where.length?"WHERE "+where.join(" AND "):""} ORDER BY r.created_a
  res.json(await q(sql,params));
});
app.get("/api/repairs/:id",auth,can("view_repairs"),async(req,res)=>{
  const r=await one(`SELECT r.*,c.name customer_name,c.phone customer_phone,c.id
    cu.full_name created_by_name,uu.full_name updated_by_name
    FROM repair_orders r JOIN customers c ON c.id=r.customer_id
    LEFT JOIN users cu ON cu.id=r.created_by LEFT JOIN users uu ON uu.id=r.update
  if(!r)return res.status(404).json({error:"إصلاح غير موجود!"});
  r.history=await q(`SELECT h.*,u.full_name changed_by_name FROM repair_status_hi
  r.payments=await q(`SELECT p.*,u.full_name recorded_by_name FROM payments p LEF
  r.parts=await q(`SELECT rp.*,sp.name part_name FROM repair_parts rp JOIN spare_
  res.json(r);
});

app.post("/api/repairs",auth,can("add_repair"),async(req,res)=>{
  const client=await pool.connect();
  try{
    await client.query("BEGIN");
    await setTenant(client,req.user.shop_id);
    const b=req.body;
    const customerName=String(b.customerName||"عميل").trim()||"عميل";
    const phone=String(b.phone||"").trim();
    let customerId;
    // Never rename an existing customer when creating a new repair.
    // Empty phone numbers never match another customer.
    if(phone){
      const c=await client.query("SELECT id FROM customers WHERE phone=$1 ORDER B
      if(c.rows[0]) customerId=c.rows[0].id;
      else customerId=(await client.query("INSERT INTO customers(shop_id,name,pho
    }else{
      customerId=(await client.query("INSERT INTO customers(shop_id,name,phone) V
    }
```

```
    await client.query("SELECT pg_advisory_xact_lock(1842026)");
    const last=await client.query("SELECT COALESCE(MAX(receipt_no),184)+1 n FROM
    const no=last.rows[0].n, tracking="QF-"+String(no).padStart(4,"0");
    const r=(await client.query(`INSERT INTO repair_orders(shop_id,receipt_no,tra
      VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,'19$,19$,18$,17$,16$,
      [req.user.shop_id,no,tracking,customerId,b.brand,b.model,b.color,b.power,b.
    await client.query("INSERT INTO repair_status_history(shop_id,repair_id,statu
    await client.query("COMMIT");
    await log(req.user.id,"إضافة وصل",r.id,{receipt_no:no});
    res.status(201).json(r);
  }catch(e){await client.query("ROLLBACK");console.error(e);res.status(500).json(
});

app.patch("/api/repairs/:id",auth,can("edit_repair"),async(req,res)=>{
  const b=req.body;
  const client=await pool.connect();
  try{
    await client.query("BEGIN");
    await setTenant(client,req.user.shop_id);
    const existing=await client.query("SELECT * FROM repair_orders WHERE id=$1 FO
    if(!existing.rows[0]){await client.query("ROLLBACK");return res.status(404).j
    const old=existing.rows[0];
    let customerId=old.customer_id;
    if(b.customerName!==undefined || b.phone!==undefined){
      const oldCustomer=(await client.query("SELECT id,name,phone FROM customers
      const newName=String(b.customerName===undefined?(oldCustomer?.name||"عميل")
      const newPhone=String(b.phone===undefined?(oldCustomer?.phone||""):b.phone)
      const oldName=String(oldCustomer?.name||"").trim();
      const oldPhone=String(oldCustomer?.phone||"").trim();
      if(newName!==oldName || newPhone!==oldPhone){
        // Never mutate a shared customer record when editing one repair.
        const same=await client.query("SELECT id FROM customers WHERE name=$1 AND
        if(same.rows[0]) customerId=same.rows[0].id;
        else customerId=(await client.query("INSERT INTO customers(shop_id,name,p
      }
    }
    const r=(await client.query(`UPDATE repair_orders SET
      customer_id=$1, brand=$2, model=$3, color=$4, due_at=COALESCE($5,due_at),
      power_state=$6, fault=$7, diagnosis=$8, expected_price=COALESCE($9,expected
      part_cost=COALESCE($11,part_cost), accessories=COALESCE($12,accessories), a
      notes=COALESCE($14,notes), supplier=COALESCE($15,supplier), updated_by=$16,
      [customerId,b.brand,b.model,b.color,b.dueAt===undefined?null:b.dueAt,b.powe
       b.price===undefined?null:Number(b.price),b.paid===undefined?null:Number(b.
       b.accessories==null?null:JSON.stringify(b.accessories),b.accessoryNotes,b.
    await client.query("COMMIT");
    await log(req.user.id,"تعديل بيانات الوصل",r.id,{changed_fields:Object.keys(b
    res.json(r);
```

```
  }catch(e){await client.query("ROLLBACK");console.error(e);res.status(500).json(
});

app.get("/api/repairs/:id/parts",auth,can("view_repairs"),async(req,res)=>{
  res.json(await q(`SELECT rp.*,sp.name part_name FROM repair_parts rp JOIN spare
});
app.post("/api/repairs/:id/parts",auth,can("edit_repair"),async(req,res)=>{
  const name=String(req.body?.name||'').trim(); const qty=Math.max(1,Number(req.b
  if(!name)return res.status(400).json({error:'اسم القطعة مطلوب'});
  const client=await pool.connect(); try{await client.query('BEGIN');await setTen
    const sp=(await client.query(`INSERT INTO spare_parts(shop_id,name,quantity,c
    const rp=(await client.query(`INSERT INTO repair_parts(shop_id,repair_id,part
    await client.query(`UPDATE repair_orders SET part_cost=COALESCE(part_cost,0)+
    await client.query('COMMIT'); res.status(201).json({...rp,part_name:name});
  }catch(e){await client.query('ROLLBACK');console.error(e);res.status(500).json(
});
app.delete("/api/repairs/:id/parts/:partId",auth,can("edit_repair"),async(req,res
app.delete("/api/repairs/:id",auth,can("delete_repairs"),async(req,res)=>{
  try{
    const r=await one("SELECT id,receipt_no,tracking_code FROM repair_orders WHER
    if(!r)return res.status(404).json({error:"الوصل غير موجود"});
    await log(req.user.id,"حذف الوصل",r.id,{receipt_no:r.receipt_no,tracking_code
    await pool.query("DELETE FROM repair_orders WHERE id=$1",[r.id]);
    res.json({ok:true});
  }catch(e){console.error(e);res.status(500).json({error:"تعذر حذف الوصل"})}
});

app.patch("/api/repairs/:id/status",auth,can("change_status"),async(req,res)=>{
  const status=req.body.status;
  if(!STATUSES.includes(status))return res.status(400).json({error:"الة غير صالحة
  const r=await one("UPDATE repair_orders SET status=$1,updated_by=$2,updated_at=
  if(!r)return res.status(404).json({error:"غير موجود"});
  await q("INSERT INTO repair_status_history(shop_id,repair_id,status,changed_by)
  let whatsapp=null;
  if(status==='تم اصلاحه'){
    const phone=(await one('SELECT phone FROM customers WHERE id=$1',[r.customer_
    const msg=`مرحبًا، نعلمكم أن هاتفكم في BN SMART جاهزًا للاستلام.\nرقم الوصل:
    await q(`INSERT INTO notifications(shop_id,repair_id,channel,message,created_
    const digits=String(phone).replace(/\D/g,'');
    const intl=digits.startsWith('0')?'213'+digits.slice(1):digits;
    whatsapp={phone:intl,message:msg,url:`https://wa.me/${intl}?text=${encodeURIC
  }
  await log(req.user.id,"تغيير حالة الإصلاح",r.id,{status});
  res.json({...r,whatsapp});
});

app.patch("/api/repairs/:id/received",auth,can("edit_repair"),async(req,res)=>{
```

```
  try{
    const raw=req.body?.received;
    const received = raw===true || raw==="true" || raw===1 || raw==="1";
    const r=await one(
      `UPDATE repair_orders
       SET customer_received=$1,
           received_at=CASE WHEN $1 THEN COALESCE(received_at,NOW()) ELSE NULL EN
           updated_by=$2, updated_at=NOW()
       WHERE id=$3 RETURNING *`,
      [received,req.user.id,req.params.id]
    );
    if(!r)return res.status(404).json({error:"الوصل غير موجود"});
    // Logging must never turn a successful status update into a 500 response.
    try{
      await log(req.user.id,received?"إلغاء تأكيد استلام الجهاز":"از من طرف الزبون"
    }catch(logErr){ console.error("pickup log error:",logErr); }
    res.json(r);
  }catch(e){
    console.error("pickup update error:",e);
    res.status(500).json({error:"تعذر تحديث حالة الاستلام",detail:process.env.NODE_
  }
});

app.post("/api/repairs/:id/payments",auth,can("edit_repair"),async(req,res)=>{
  const amount=Number(req.body.amount)||0;
  if(amount<=0)return res.status(400).json({error:"قيمة الدفعة غير صحيحة"});
  const client=await pool.connect();
  try{
    await client.query("BEGIN");
    await setTenant(client,req.user.shop_id);
    const r=(await client.query("UPDATE repair_orders SET paid_amount=paid_amount
    if(!r){await client.query("ROLLBACK");return res.status(404).json({error:"جود
    await client.query("INSERT INTO payments(shop_id,repair_id,amount,recorded_by
    await client.query("COMMIT");await log(req.user.id,"تسجيل دفعة",r.id,{amount}
  }catch(e){await client.query("ROLLBACK");res.status(500).json({error:"يل الدفعة
});

app.get("/api/customers",auth,can("customers"),async(req,res)=>{
  res.json(await q(`SELECT c.id,c.name,c.phone,COUNT(r.id)::int repairs_count,COA
  FROM customers c LEFT JOIN repair_orders r ON r.customer_id=c.id GROUP BY c.id
});

app.post("/api/customers",auth,can("customers"),async(req,res)=>{
  const name=String(req.body?.name||"").trim();
  const phone=String(req.body?.phone||"").trim();
  if(!name || !phone) return res.status(400).json({error:"بون ورقم الهاتف مطلوبان
  try{
```

```
    const existing=await one("SELECT id,name,phone FROM customers WHERE shop_id=$
    if(existing) return res.status(409).json({error:"لهاتف مسجل مسبقًا لهذا الزبون
    const c=await one("INSERT INTO customers(shop_id,name,phone) VALUES($1,$2,$3)
    await log(req.user.id,"إضافة زبون",null,{customer_id:c.id});
    res.status(201).json(c);
  }catch(e){console.error(e);res.status(500).json({error:"تعذر إضافة الزبون"})}
});

app.post("/api/auth/change-password", auth, async(req,res)=>{
  const {currentPassword,newPassword}=req.body||{};
  if(!currentPassword || !newPassword) return res.status(400).json({error:"لجديدة
  if(String(newPassword).length < 8) return res.status(400).json({error:"على الأقل
  const u=await one("SELECT * FROM users WHERE id=$1",[req.user.id]);
  if(!u || !u.active || !(await bcrypt.compare(String(currentPassword),u.password
  const hash=await bcrypt.hash(String(newPassword),12);
  await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2",[hash,u.id]);
  await log(req.user.id,"تغيير كلمة المرور",null);
  res.json({ok:true});
});

app.get("/api/users",auth,manager,async(req,res)=>{
  res.json((await q("SELECT id,username,full_name,phone,role,active,permissions,l
});
app.post("/api/users",auth,manager,async(req,res)=>{
  const b=req.body;if(!b.username||!b.fullName||!b.password)return res.status(400
  const hash=await bcrypt.hash(b.password,12);
  const permissions={...DEFAULT_PERMS,...(b.permissions||{})};
  try{
    const u=await one("INSERT INTO users(shop_id,username,full_name,phone,passwor
      [req.user.shop_id,b.username,b.fullName,b.phone||"",hash,JSON.stringify(per
    await log(req.user.id,"إضافة متعاون",null,{username:b.username});res.status(2
  }catch(e){res.status(409).json({error:"اسم المستخدم مستخدم بالفعل"})}
});
app.patch("/api/users/:id",auth,manager,async(req,res)=>{
  const b=req.body;let u=await one("SELECT * FROM users WHERE id=$1",[req.params.
  const username=b.username==null?u.username:String(b.username).trim();
  if(!username)return res.status(400).json({error:"اسم المستخدم مطلوب"});
  const role=b.role==null?u.role:String(b.role);
  if(!["manager","collaborator"].includes(role))return res.status(400).json({erro
  const active=b.active==null?u.active:Boolean(b.active);
  if((u.role!==role || u.active!==active) && (u.role==="manager") && (role!=="man
    const count=Number((await one("SELECT COUNT(*)::int n FROM users WHERE role='
    if(count<=1)return res.status(400).json({error:"يبقى مدير واحد فعّال على الأقل
  }
  const perms=role==="manager"?JSON.stringify(Object.fromEntries(Object.keys(DEFA
  const passwordHash=b.password?await bcrypt.hash(b.password,12):u.password_hash;
  try{
```

```
    u=await one("UPDATE users SET username=$1,full_name=COALESCE($2,full_name),ph
      [username,b.fullName,b.phone,role,active,perms,passwordHash,req.params.id])
    await log(req.user.id,"تعديل حساب مستخدم",null,{user_id:u.id,username:u.usern
  }catch(e){
    if(e && e.code==="23505") return res.status(409).json({error:"م مستخدم بالفعل
    console.error(e);res.status(500).json({error:"تعذر تعديل الحساب"});
  }
});
app.post("/api/users/:id/password",auth,manager,async(req,res)=>{
  const b=req.body||{};
  if(!b.newPassword || String(b.newPassword).length<8)return res.status(400).json
  const u=await one("SELECT id,username,role FROM users WHERE id=$1",[req.params.
  if(!u)return res.status(404).json({error:"المستخدم غير موجود"});
  const hash=await bcrypt.hash(String(b.newPassword),12);
  await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2",[hash,u.id]);
  await log(req.user.id,"تغيير كلمة مرور مستخدم",null,{user_id:u.id,username:u.us
  res.json({ok:true});
});
app.put("/api/users/:id/password",auth,manager,async(req,res)=>{
  const b=req.body||{};
  if(!b.newPassword || String(b.newPassword).length<8)return res.status(400).json
  const u=await one("SELECT id,username,role FROM users WHERE id=$1",[req.params.
  if(!u)return res.status(404).json({error:"المستخدم غير موجود"});
  try{
    const hash=await bcrypt.hash(String(b.newPassword),12);
    await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2",[hash,u.id])
    await log(req.user.id,"تغيير كلمة مرور مستخدم",null,{user_id:u.id,username:u.
    res.json({ok:true});
  }catch(e){console.error(e);res.status(500).json({error:"تعذر تغيير كلمة المرور"
});

app.delete("/api/users/:id",auth,manager,async(req,res)=>{
  if(Number(req.params.id)===req.user.id)return res.status(400).json({error:"حالي
  const client=await pool.connect();
  try{
    await client.query("BEGIN");
    await setTenant(client,req.user.shop_id);
    const u=(await client.query("SELECT id,username,full_name,role FROM users WHE
    if(!u)return res.status(404).json({error:"المستخدم غير موجود"});
    if(u.role==="manager") {
      const managers=Number((await client.query("SELECT COUNT(*)::int n FROM user
      if(managers<1)return res.status(403).json({error:"أن يوجد مدير آخر فعّال أولاً
    }
    const label=`${u.full_name} (${u.username})`;
    await client.query("UPDATE activity_logs SET user_id=NULL, details=jsonb_set(
    await client.query("UPDATE repair_orders SET created_by=NULL, updated_by=NULL
    await client.query("UPDATE repair_status_history SET changed_by=NULL WHERE ch
```

```
    await client.query("UPDATE payments SET recorded_by=NULL WHERE recorded_by=$1
    await client.query("UPDATE repair_parts SET added_by=NULL WHERE added_by=$1",
    await client.query("UPDATE inventory_transactions SET performed_by=NULL WHERE
    await client.query("UPDATE notifications SET created_by=NULL WHERE created_by
    await client.query("DELETE FROM users WHERE id=$1",[u.id]);
    await client.query("INSERT INTO activity_logs(shop_id,user_id,action,details)
    await client.query("COMMIT");
    res.json({ok:true});
  }catch(e){await client.query("ROLLBACK");console.error(e);res.status(500).json(
  finally{client.release()}
});
app.get("/api/activity",auth,manager,async(req,res)=>{
  res.json(await q(`SELECT a.*,u.full_name user_name,r.receipt_no FROM activity_l
});

const SUPPLIER_NAMES=["BN SMART","MONTEL","AABIDIN","HASSAN","RAHT ELBAL"];
const SUPPLIER_ITEMS=["LCD","BAT","NAP CHARGE","GLASS","SERSOU","CONCTOUR"];
app.get("/api/suppliers",auth,can("inventory"),async(req,res)=>{
  try{const rows=await q(`SELECT supplier,COUNT(*)::int purchases_count,COALESCE(
});
app.get("/api/suppliers/:supplier/purchases",auth,can("inventory"),async(req,res)
app.post("/api/suppliers/purchases",auth,can("inventory"),async(req,res)=>{const
app.delete("/api/suppliers/purchases/:id",auth,manager,async(req,res)=>{try{const

app.get("/api/settings",auth,can("settings"),async(req,res)=>{
  const value=(await one("SELECT value FROM settings WHERE key='shop'"))?.value||
  res.json({...value,name:"BN SMART"});
});
app.put("/api/settings",auth,manager,async(req,res)=>{
  const current=(await one("SELECT value FROM settings WHERE key='shop'"))?.value
  const next={...current,...req.body,name:"BN SMART"};
  await pool.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',$2)
  await log(req.user.id,"تحديث إعدادات المحل");res.json(next);
});

app.get("/api/backup",auth,manager,async(req,res)=>{
  const data={customers:await q("SELECT * FROM customers"),repairs:await q("SELEC
  res.json({exported_at:new Date().toISOString(),data});
});

app.get("/api/track/:code",async(req,res)=>{
  const r=await one(`SELECT * FROM public_track_repair($1)`,[req.params.code]);
  if(!r)return res.status(404).json({error:"رقم التتبع غير موجود"});
  const history=await one(`SELECT public_track_history($1) AS rows`,[r.id]);
  const historyRows=history?.rows||[];
  res.json({receipt_no:r.receipt_no,tracking_code:r.tracking_code,device:[r.brand
});
```

```
app.get("/api/qr/:code",async(req,res)=>{try{const base=`${req.protocol}://${req.

app.get("/track/:code",(req,res)=>res.sendFile(path.join(__dirname,"public","trac
app.get("/share/:code",(req,res)=>{res.set("Cache-Control","no-store, no-cache, m
app.get("*",(req,res)=>res.sendFile(path.join(__dirname,"public","index.html")));

init().then(()=>app.listen(PORT,()=>console.log(`BN SMART running on http://local
```